

SIMATS ENGINEERING

ASSESSMENT TOOL 8

COURSE NAME: Database Management Systems

COURSE CODE: CSA0506

COURSE OUTCOME COVERED

CO3: Analyze relational databases using functional dependencies, normalization(1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.(BL2, BL3)

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

Code of Conduct:

I G.JEEVESH(192511440) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:
G.Jeevesh**

1. Online Shopping Platform — Normalize to 3NF

Relation:

```
Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)
```

Functional Dependencies

- $OrderID \rightarrow CustID$
- $CustID \rightarrow CustName$
- $ProdID \rightarrow ProdName, Price$
- $(OrderID, ProdID) \rightarrow Qty$

Candidate Key: $(OrderID, ProdID)$

2NF Decomposition

Remove partial dependencies:

- `Customer(CustID, CustName)`
- `Product(ProdID, ProdName, Price)`
- `Order(OrderID, CustID)`
- `OrderItem(OrderID, ProdID, Qty)`

3NF

There is a transitive dependency $OrderID \rightarrow CustID \rightarrow CustName$, which is removed by the above decomposition.

Final 3NF relations:

- `Customer(CustID, CustName)`
 - `Product(ProdID, ProdName, Price)`
 - `Orders(OrderID, CustID)`
 - `OrderItem(OrderID, ProdID, Qty)`
-

2. Hospital — Normalize to BCNF

Relation:

```
Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)
```

Functional Dependencies

- $PID \rightarrow PName, DocID, WardNo$
- $DocID \rightarrow DocName, Specialization$
- $WardNo \rightarrow WardName$

Candidate Key: PID

Anomalies

- **Update:** Doctor information must be changed in many patient records.
- **Insertion:** Cannot store doctor/ward information without a patient.
- **Deletion:** Removing the last patient may remove doctor/ward information.

BCNF Decomposition

- $Patient(PID, PName, DocID, WardNo)$
- $Doctor(DocID, DocName, Specialization)$
- $Ward(WardNo, WardName)$

Each determinant is a candidate key in its relation.

Hence, the schema is in BCNF.

3. University Enrollment — Apply 2NF

Relation:

$Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)$

Functional Dependencies

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$
- $(SID, CourseID) \rightarrow Grade$

Candidate Key: $(SID, CourseID)$

Partial Dependencies

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$

These violate **2NF** because non-key attributes depend on only part of the composite key.

2NF Decomposition

- `Student(SID, SName)`
- `Course(CourseID, CourseName, Faculty)`
- `Enrollment(SID, CourseID, Grade)`

Result: Partial dependencies are removed and the schema is in **2NF**.

4. Airline Booking — Normalize to BCNF

Relation:

`Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)`

Functional Dependencies

- `PassID → PassName`
- `FlightNo → DeptCity, ArrCity`
- `(FlightNo, Date, PassID) → Seat`

Candidate Key: `(FlightNo, Date, PassID)`

Decomposition

- `Passenger(PassID, PassName)`
- `Flight(FlightNo, DeptCity, ArrCity)`
- `Booking(FlightNo, Date, PassID, Seat)`

All determinants in the decomposed relations are candidate keys.

Hence, the relations satisfy BCNF.

5. HR System — Normalize to 3NF

Assume:

`Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, JobID, JobTitle)`

Functional Dependencies

- `EmpID → EmpName, DeptID, ManagerID, JobID`
- `DeptID → DeptName, ManagerID`
- `ManagerID → ManagerName`

- `JobID → JobTitle`

Transitive Dependencies

- `EmpID → DeptID → DeptName`
- `EmpID → ManagerID → ManagerName`
- `EmpID → JobID → JobTitle`

3NF Decomposition

- `Employee(EmpID, EmpName, DeptID, ManagerID, JobID)`
- `Department(DeptID, DeptName, ManagerID)`
- `Manager(ManagerID, ManagerName)`
- `Job(JobID, JobTitle)`

Result: Partial and transitive dependencies are removed; the design is in **3NF**.

6. Library — Decompose to 4NF

Relation:

`Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)`

Assume a member can independently have multiple **books** and multiple **genres**.

Functional Dependencies

- `MemberID → MemberName`
- `BookID → Title, Author`

Multivalued Dependencies

- `MemberID ↔ BookID`
- `MemberID ↔ Genre`

These independent multivalued facts can cause redundancy and violate **4NF**.

4NF Decomposition

- `Member(MemberID, MemberName)`
- `Book(BookID, Title, Author)`
- `MemberBook(MemberID, BookID)`
- `MemberGenre(MemberID, Genre)`

- `Borrow(MemberID, BookID, BorrowDate)`

Result: Independent multivalued dependencies are separated, satisfying **4NF**.

7. Telecom Billing — Keys and FDs

Assume:

`Billing(CustomerID, PhoneNo, CustomerName, PlanID, PlanName, CallID, CallDuration, Charge)`

Functional Dependencies

- `CustomerID → CustomerName`
- `PhoneNo → CustomerID, PlanID`
- `PlanID → PlanName`
- `CallID → PhoneNo, CallDuration`
- `(CallID) → Charge`

Candidate Keys

`CallID` can uniquely identify a call record.

Candidate Key: `CallID`

Primary Key: `CallID`

Important Dependencies

- `CallID → PhoneNo`
- `PhoneNo → CustomerID`
- `CustomerID → CustomerName`
- `PhoneNo → PlanID`
- `PlanID → PlanName`

This reveals transitive dependencies such as:

`CallID → PhoneNo → CustomerID → CustomerName`

These should be removed during normalization.

8. Retail Inventory — Complete Normalization

Relation:

`Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)`

Functional Dependencies

- `PID → PName, SupplierID, Category, Price, Warehouse`
- `SupplierID → SupplierName`

Transitive Dependency

`PID → SupplierID → SupplierName`

2NF

Since `PID` is a single-attribute key, there are no partial dependencies.

3NF Decomposition

- `Product(PID, PName, SupplierID, Category, Price, Warehouse)`
- `Supplier(SupplierID, SupplierName)`

Final Design

Product

`PID, PName, SupplierID, Category, Price, Warehouse`

Supplier

`SupplierID, SupplierName`

The design eliminates redundancy and transitive dependency and is in **3NF**. Under the stated FDs, it also satisfies BCNF.

9. School ERP — Normalize to BCNF

Assume the poorly normalized relation is:

`School(SID, SName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)`

Functional Dependencies

- `SID → SName`
- `ClassID → ClassName, TeacherID`
- `TeacherID → TeacherName`
- `SubjectID → SubjectName`
- `(SID, SubjectID) → Grade`

Problems

- Student details are repeated.
- Teacher details are repeated for every class.
- Subject details are repeated.
- Update, insertion, and deletion anomalies occur.

BCNF Decomposition

- `Student(SID, SName, ClassID)`
- `Class(ClassID, ClassName, TeacherID)`
- `Teacher(TeacherID, TeacherName)`
- `Subject(SubjectID, SubjectName)`
- `Enrollment(SID, SubjectID, Grade)`

Each determinant is a key in its corresponding relation.

Hence, the design is normalized up to BCNF.

10. Movie Streaming Platform — Normalize to 5NF

Assume:

`Streaming(MovieID, ActorID, DirectorID, PlatformID)`

Suppose the system stores independent relationships:

- `Movie ↔ Actor`
- `Movie ↔ Director`
- `Movie ↔ Platform`

These can create a **join dependency (JD)**.

Decomposition

- `MovieActor(MovieID, ActorID)`
- `MovieDirector(MovieID, DirectorID)`
- `MoviePlatform(MovieID, PlatformID)`

5NF

Each relation contains one independent relationship. Joining the three relations reconstructs the original valid combinations **when the business rules guarantee that the relationships are independent.**